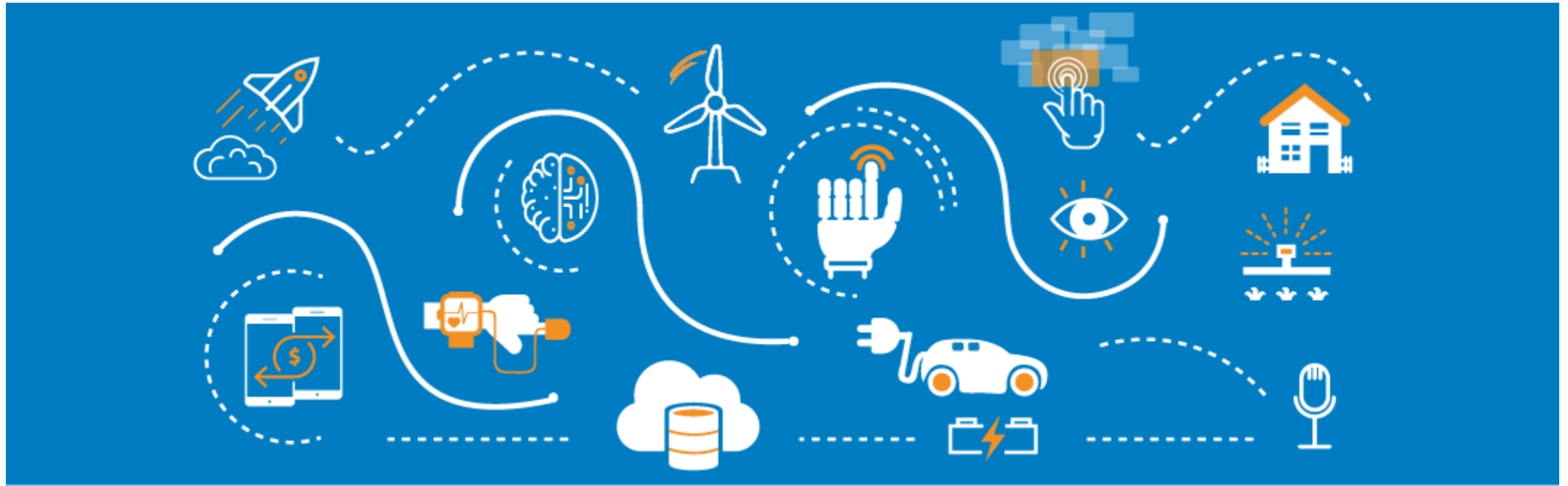




Computer Vision Fundamentals Part 2

Kien Tran

Contents

1

Image Processing

2

Video Basics with Python and OpenCV

3

Object Detection with Python and OpenCV Part 1

4

Object Detection with Python and OpenCV Part 2

5

Demo Applications using Computer Vision (extend)

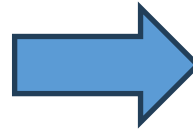
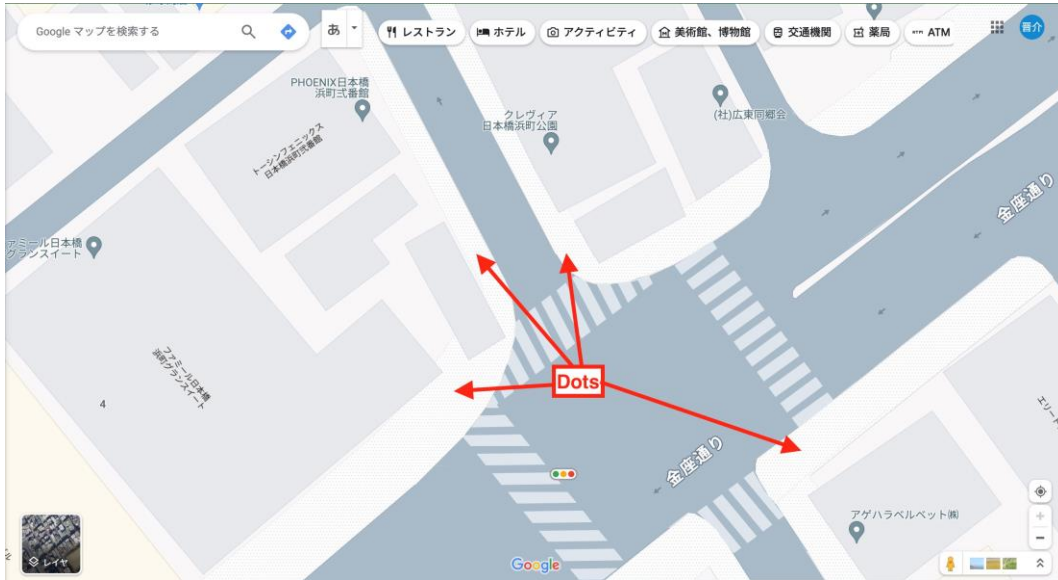
1. Image Processing

1.1. Image Thresholding

- Thresholding is fundamentally a very simple method of segmenting an image into different parts
- Thresholding will convert an image to consist of only two value: white & black

1. Image Processing

1.1. Image Thresholding



1. Image Processing

1.1. Image Thresholding

Binary

$$\text{dst}(x, y) = \begin{cases} \text{maxval} & \text{if } \text{src}(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$$

Inverted Binary

$$\text{dst}(x, y) = \begin{cases} 0 & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{maxval} & \text{otherwise} \end{cases}$$

Truncated

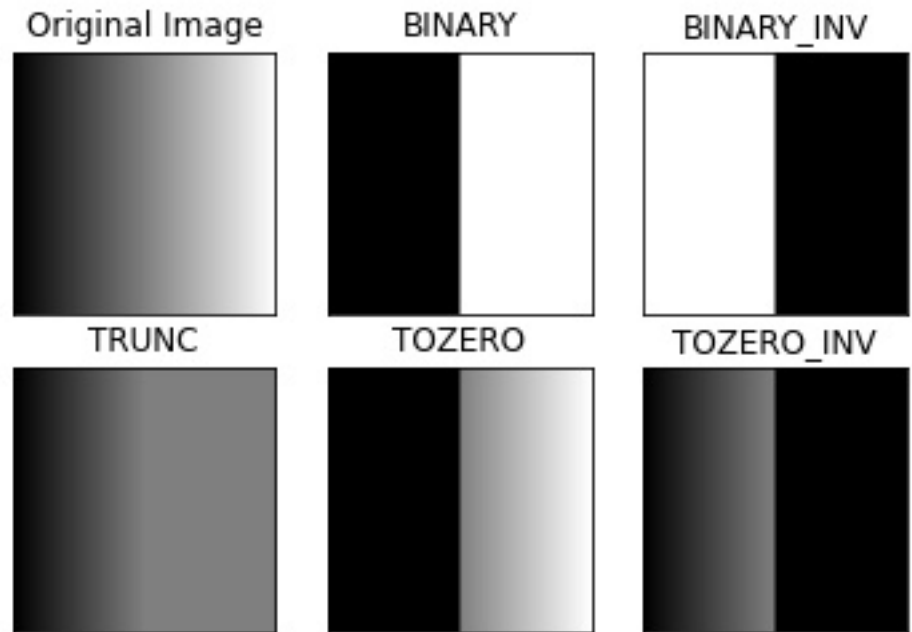
$$\text{dst}(x, y) = \begin{cases} \text{threshold} & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{src}(x, y) & \text{otherwise} \end{cases}$$

To Zero

$$\text{dst}(x, y) = \begin{cases} \text{src}(x, y) & \text{if } \text{src}(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$$

To Zero Inverted

$$\text{dst}(x, y) = \begin{cases} 0 & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{src}(x, y) & \text{otherwise} \end{cases}$$



image

1. Image Processing

1.1. Image Thresholding

Demo: 03-Blurring-and-Smoothing.ipynb

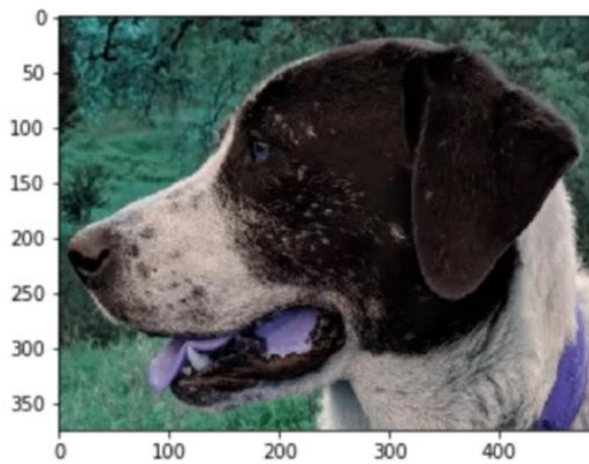
1. Image Processing

1.2. Blurring or Smoothing

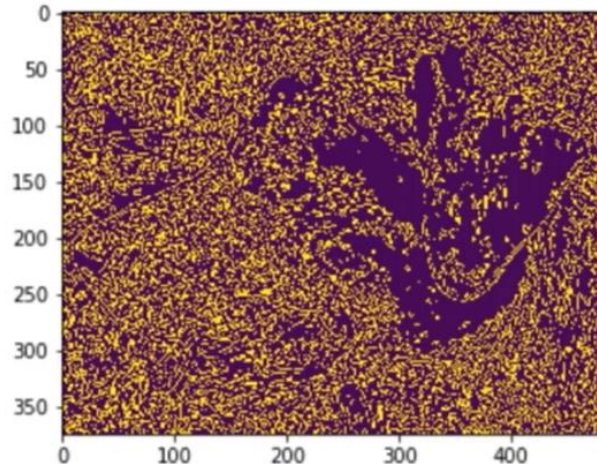
- A common operation for image processing is blurring or smoothing an image
- Smoothing an image can help get rid of noise, or help an application focus on general details
- Is combined with edge detection
- Edge detection algorithms detect too many edges when shown a high resolution image without any blurring

1. Image Processing

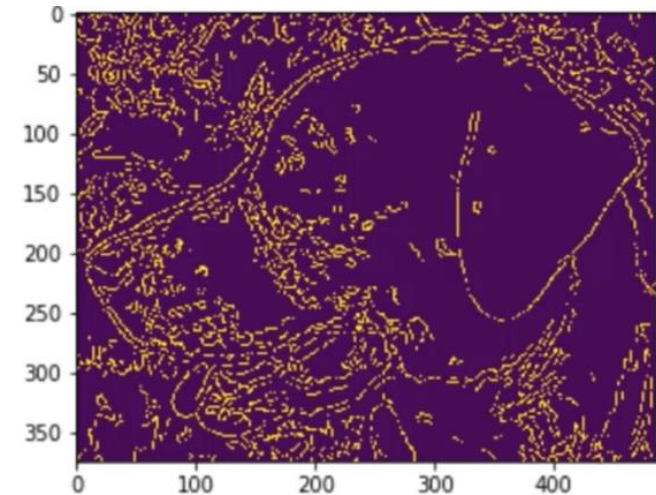
1.2. Blurring or Smoothing



Original



Detected Edges (no blur)



Detected Edges (with blur)

1. Image Processing

1.2. Blurring or Smoothing

Methods:

- Gamma Correction: make image appear brighter or darker depending on the Gamma value



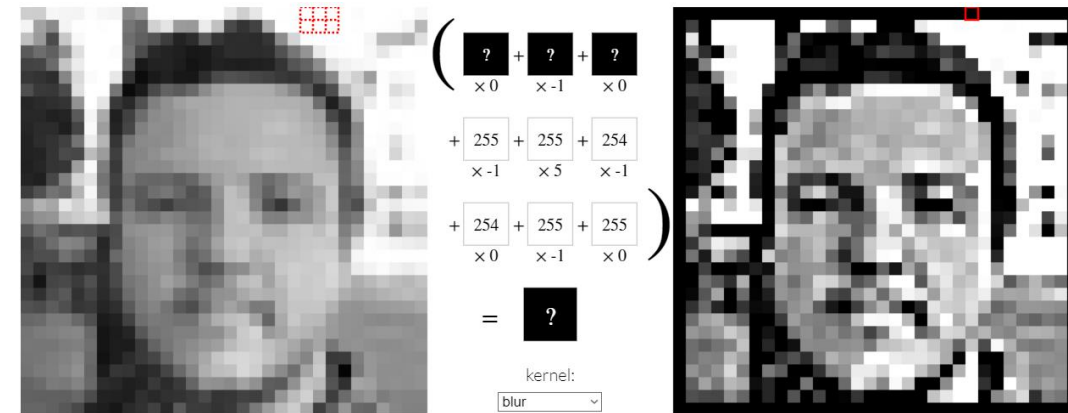
1. Image Processing

1.2. Blurring or Smoothing

Methods:

Kernel Based Filters

- Kernel can apply over an image to produce a variety of effects
- The best way to explain this is through an interactive visualization : <https://setosa.io/ev/image-kernels/>



1. Image Processing

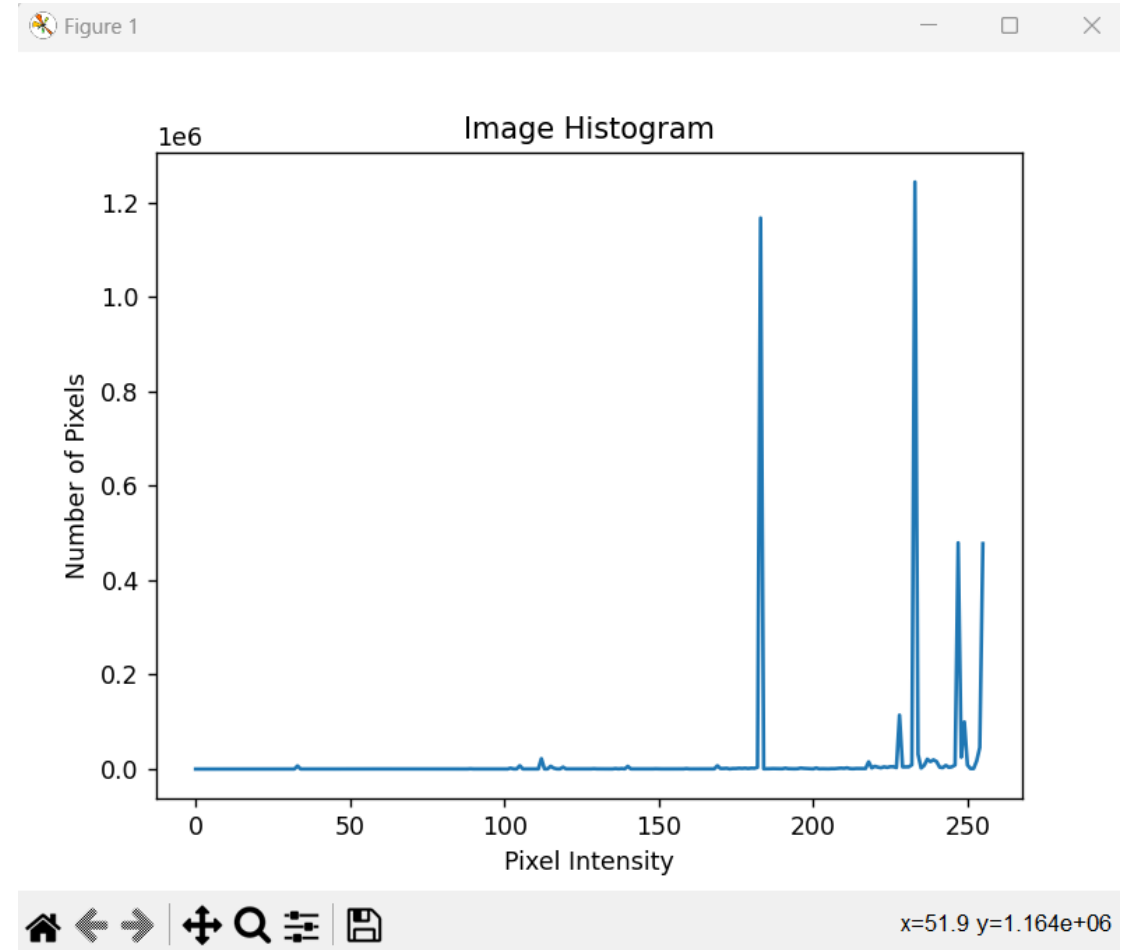
1.2. Blurring or Smoothing

Demo: 03-Blurring-and-Smoothing.ipynb

1. Image Processing

1.3. Histograms

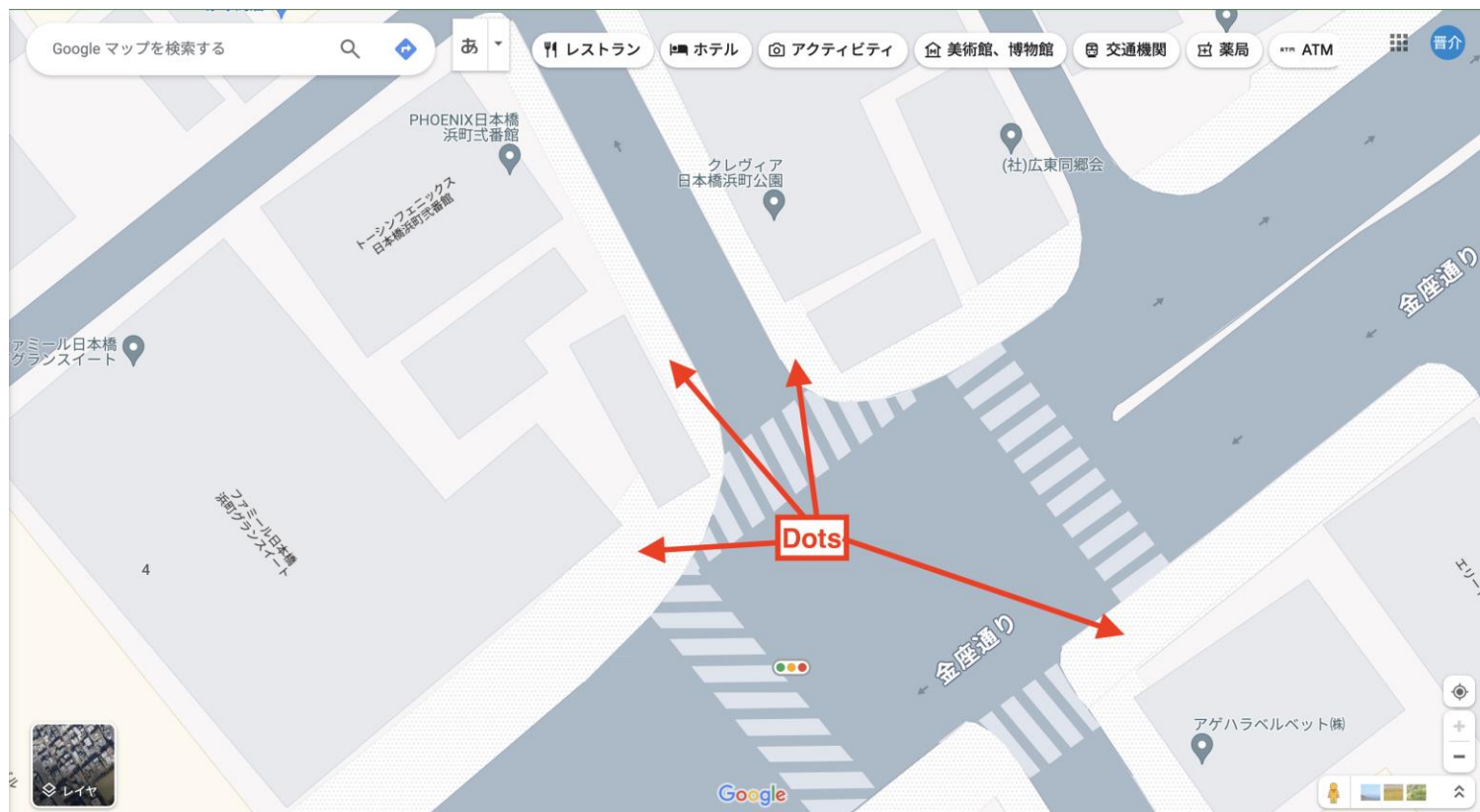
- Can display the frequency of values for colors
- Each of the three RGB channels has values between 0-255



1. Image Processing

1.3. Histograms

Demo: Detect sidewalks on map
(file analyze_histogram.py, test_map.py)



2. Video Basics with OpenCV

2.1. Introduction

Section goals:

- Use OpenCV to open a video file, webcam, camera rtsp
- Draws shapes on video
- Interact with video

2. Video Basics with OpenCV

2.2. Demo read and write video

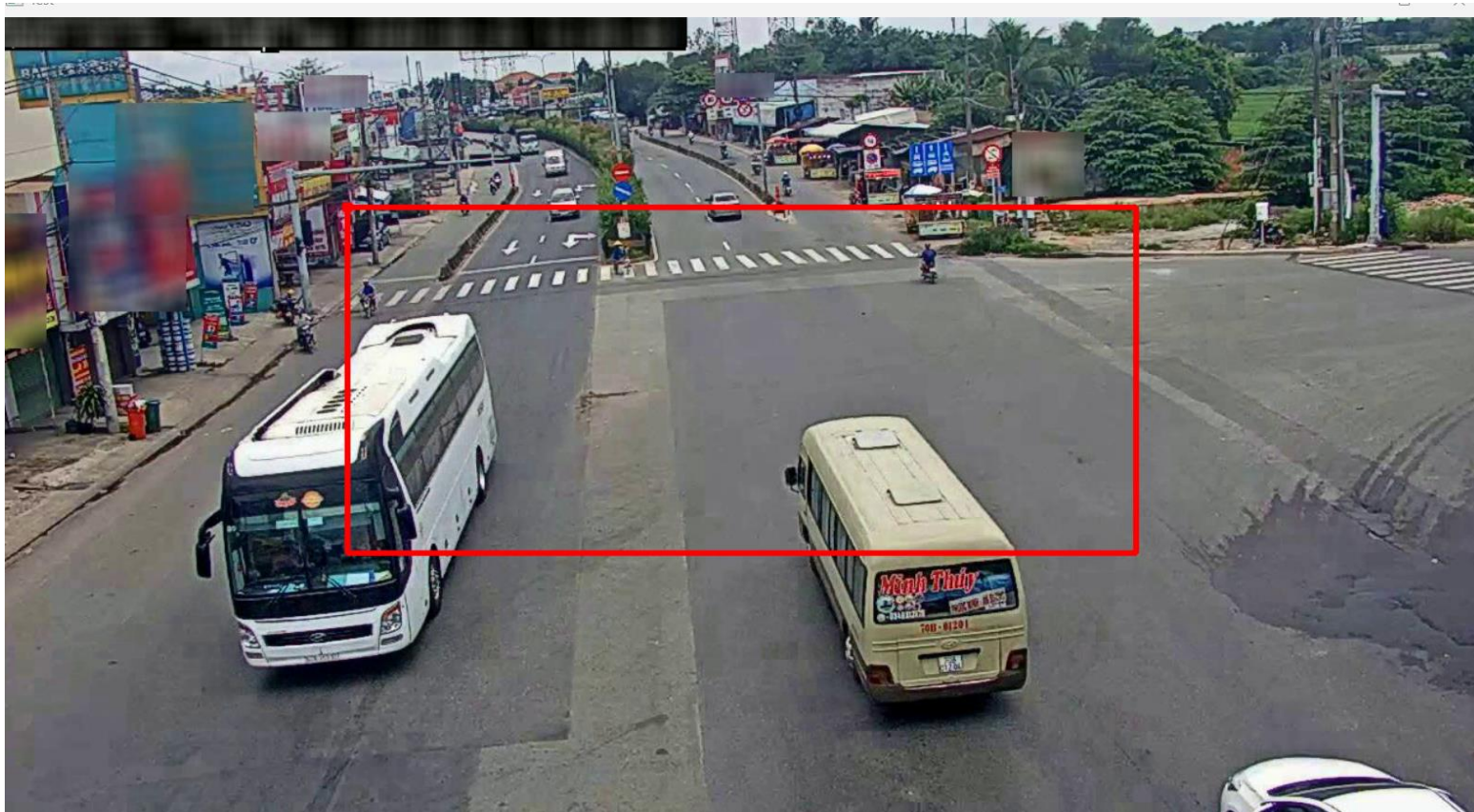
Demo on video about traffic congestion in HCM city road



2. Video Basics with OpenCV

2.3. Demo draw on video

Demo on video about traffic congestion in HCM city road



2. Video Basics with OpenCV

2.4. Assessments

[03-Video-Basics-Assessment](#)

3. Object Detection With OpenCV Part1

3.1. Introduction

Section goals:

- Understand a variety of object detection methods
- We will slowly build up to more complex methods as we go along
- Template matching: simply looking for an exact copy of an image in another image
- Corner Detection: looking for corners in image

3. Object Detection With OpenCV Part1

3.2. Template Matching

<code>cv2.TM_SQDIFF</code>	$R(x, y) = \sum_{x', y'} (T(x', y') - I(x + x', y + y'))^2$
<code>cv2.TM_SQDIFF_NORMED</code>	$R(x, y) = \frac{\sum_{x', y'} (T(x', y') - I(x + x', y + y'))^2}{\sqrt{\sum_{x', y'} T(x', y')^2 \cdot \sum_{x', y'} I(x + x', y + y')^2}}$
<code>cv2.TM_CCORR</code>	$R(x, y) = \sum_{x', y'} (T(x', y') \cdot I(x + x', y + y'))$
<code>cv2.TM_CCORR_NORMED</code>	$R(x, y) = \frac{\sum_{x', y'} (T(x', y') \cdot I(x + x', y + y'))}{\sqrt{\sum_{x', y'} T(x', y')^2 \cdot \sum_{x', y'} I(x + x', y + y')^2}}$
<code>cv2.TM_CCOEFF</code>	$R(x, y) = \sum_{x', y'} (T'(x', y') \cdot I'(x + x', y + y'))$ where, $T'(x', y') = T(x', y') - 1/(w \cdot h) \cdot \sum_{x'', y''} T(x'', y'')$ $I'(x + x', y + y') = I(x + x', y + y') - 1/(w \cdot h) \cdot \sum_{x'', y''} I(x + x'', y + y'')$
<code>cv2.TM_CCOEFF_NORMED</code>	$R(x, y) = \frac{\sum_{x', y'} (T'(x', y') \cdot I'(x + x', y + y'))}{\sqrt{\sum_{x', y'} T'(x', y')^2 \cdot \sum_{x', y'} I'(x + x', y + y')^2}}$

https://docs.opencv.org/3.4/d4/dc6/tutorial_py_template_matching.html

3. Object Detection With OpenCV Part1

3.3. Corner Detection

- A Corner is a point whose local neighborhood stands in two dominant and different edge directions
- There are various corner detection algorithms:
 - + Harris Corner Detection
 - + Shi-Tomasi Corner Detection

3. Object Detection With OpenCV Part1

3.3. Corner Detection

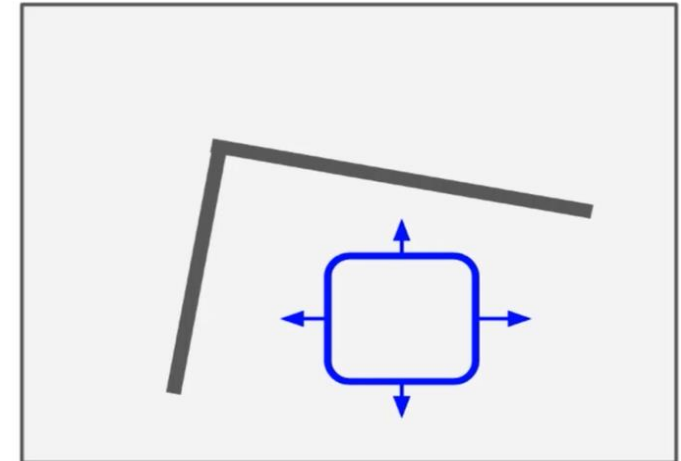
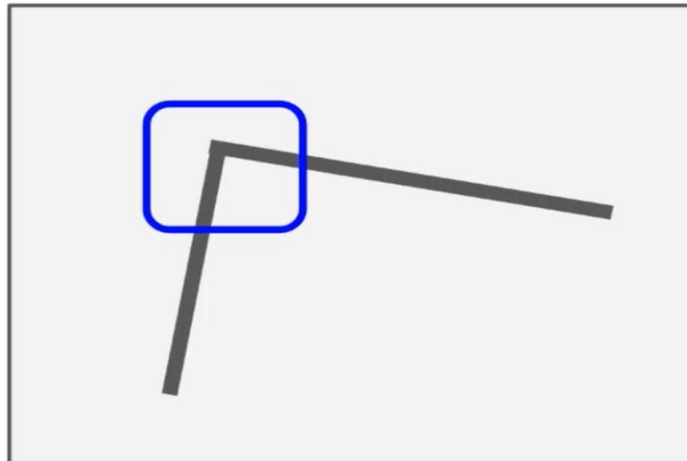
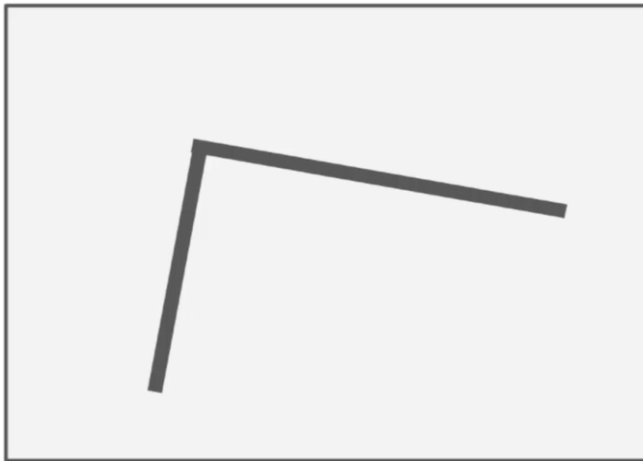
- Harris Corner Detection
 - Published in 1988 by Chris Harris and Mike Stephens
 - The basic intuition is that corners can be detected by looking for significant change in all directions

3. Object Detection With OpenCV Part1

3.3. Corner Detection

- Harris Corner Detection
- + Shifting this window in any direction would result in a large change in appearance
- + Flat regions will have no change in all directions

$$f(x, y) = \sum_{(x_k, y_k) \in W} (I(x_k, y_k) - I(x_k + \Delta x, y_k + \Delta y))^2$$



3. Object Detection With OpenCV Part1

3.3. Corner Detection

- Shi-Tomasi corner detection
 - Published in 1994 by J. Shi and C. Tomasi in their paper *Good Features to Track*
 - It made a small modification to the Harris Corner Detector which ended up with better results.
 - Harris Corner Detector uses:
 - $R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)$
 - Shi-Tomasi uses:
 - $R = \min(\lambda_1, \lambda_2)$

4. Object Detection With OpenCV Part2

4.1. Introduction

Section goals:

- Facial and Eye Detection
- Project Assessment

4. Object Detection With OpenCV Part2

4.1. Introduction Face Detection

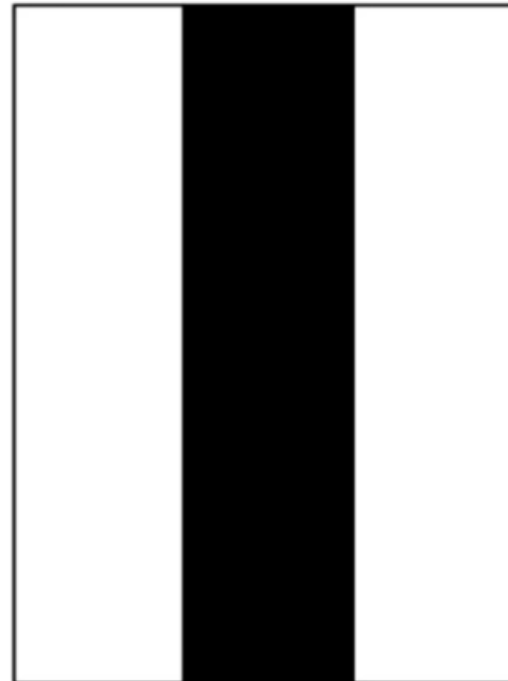
Section goals:

- Explore face detection using Haar Cascades, which is key component of the Viola-Jones object detection framework
- This is face detection NOT face **recognition**
- Will be able to very quickly detect deep learning if a face is in an image and locate it
- Need a really **large dataset** and for facial recognition

4. Object Detection With OpenCV Part2

4.2. Theory

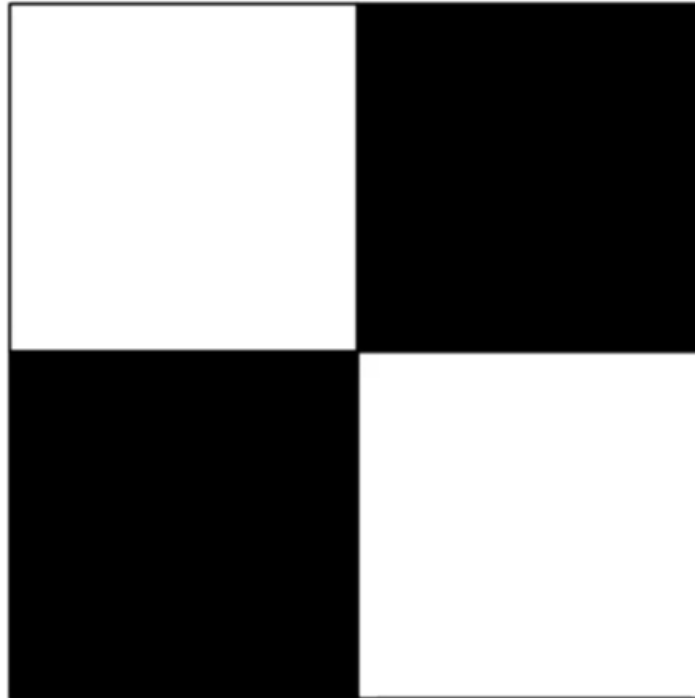
- The main feature types are:
 - Line Features



4. Object Detection With OpenCV Part2

4.2. Theory

- The main feature types are:
 - Four-rectangle Features



4. Object Detection With OpenCV Part2

4.2. Theory

- Each features is a single value obtained by subtracting sum of pixels under white rectangle from sum of pixels under black rectangle
- Realistically, my images won't be perfect edges or lines

4. Object Detection With OpenCV Part2

4.2. Theory

- These features are calculated by:

Mean (dark region) – Mean (light region)

0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1

0	0.1	0.8	1
0.3	0.1	0.7	0.8
0.1	0.2	0.8	0.8
0.2	0.2	0.8	0.8

4. Object Detection With OpenCV Part2

4.2. Theory

- $\text{sum}([0.8, 1, 0.7, 0.8, 0.8, 0.8, 0.8, 0.8]) = 6.5$
- $\text{sum}([0, 0.1, 0.3, 0.1, 0.1, 0.2, 0.2, 0.2]) = 1.2$

0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1

0	0.1	0.8	1
0.3	0.1	0.7	0.8
0.1	0.2	0.8	0.8
0.2	0.2	0.8	0.8

4. Object Detection With OpenCV Part2

4.2. Theory

- Mean = $6.5 / 8 = 0.8125$
- Mean = $1.2 / 8 = 0.15$

0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1

0	0.1	0.8	1
0.3	0.1	0.7	0.8
0.1	0.2	0.8	0.8
0.2	0.2	0.8	0.8

4. Object Detection With OpenCV Part2

4.2. Theory

- $\Delta = 0.8125 - 0.15 = 0.6625$

0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1

0	0.1	0.8	1
0.3	0.1	0.7	0.8
0.1	0.2	0.8	0.8
0.2	0.2	0.8	0.8

4. Object Detection With OpenCV Part2

4.2. Theory

- Calculating these sums for the entire image would be very computationally expensive
- The Viola-Jones algorithm solves this by using the **integral** image
- Resulting in an $O(1)$ running time of the algorithm

4. Object Detection With OpenCV Part2

4.2. Theory

- To sum up a subrectangle of its values; each coloured spot highlights the sum inside the rectangle of that colour

31	2	4	33	5	36
12	26	9	10	29	25
13	17	21	22	20	18
24	23	15	16	14	19
30	8	28	27	11	7
1	35	34	3	32	6

31	33	37	70	75	111
43	71	84	127	161	222
56	101	135	200	254	333
80	148	197	278	346	444
110	186	263	371	450	555
111	222	333	444	555	666

$$15 + 16 + 14 + 28 + 27 + 11 = 111$$
$$101 + 450 - 254 - 186 = 111$$

4. Object Detection With OpenCV Part2

4.2. Theory

- The algorithm also saves time by going through a **cascade** of classifier
- This means we will treat the image to a series (a cascade) of classifiers based on the simple features shown earlier
- Once an image fails a classifier, we can stop attempting to detect face

4. Object Detection With OpenCV Part2

4.2. Theory

- First we need a front facing image of person's face



4. Object Detection With OpenCV Part2

4.2. Theory

- The turn it to grayscale



4. Object Detection With OpenCV Part2

4.2. Theory

- Then we will begin the search for Haar Cascade Features
- One of the very first features searched for is an edge feature indicating eyes and cheeks



4. Object Detection With OpenCV Part2

4.2. Theory

- If the image were to fail for the search of this feature, we can quickly say there is no face



4. Object Detection With OpenCV Part2

4.2. Theory

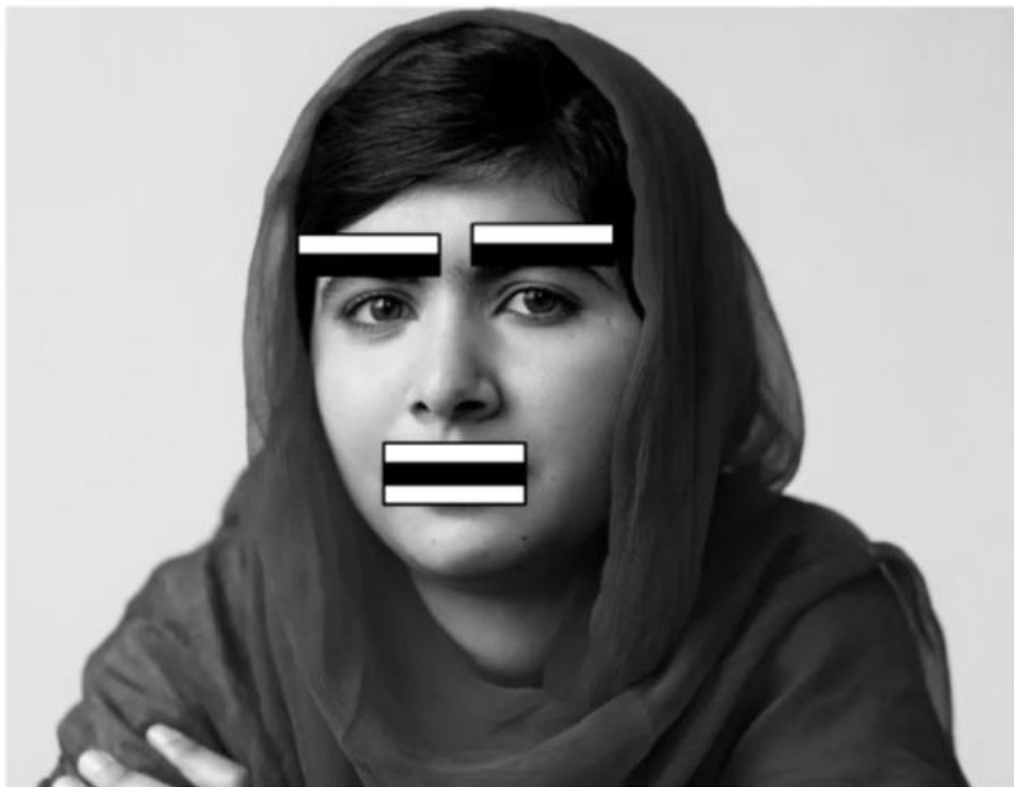
- If it passed, then we search for the next feature, such as the bridge of the nose



4. Object Detection With OpenCV Part2

4.2. Theory

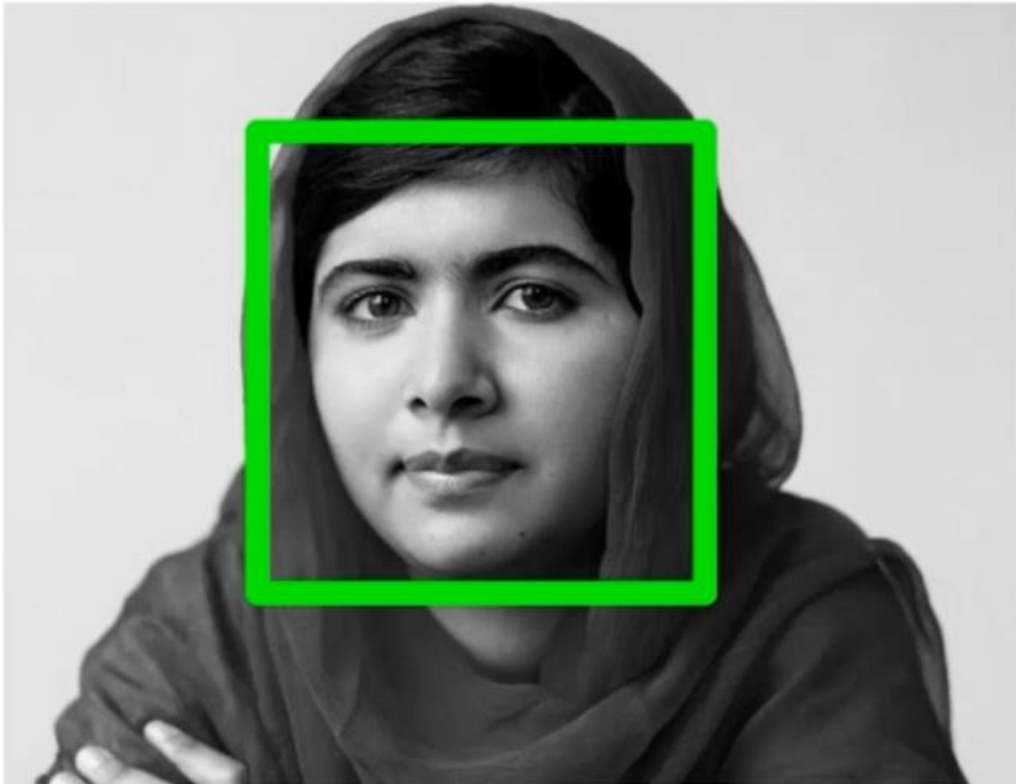
- Continue



4. Object Detection With OpenCV Part2

4.2. Theory

- Until the algorithm detects the face



4. Object Detection With OpenCV Part2

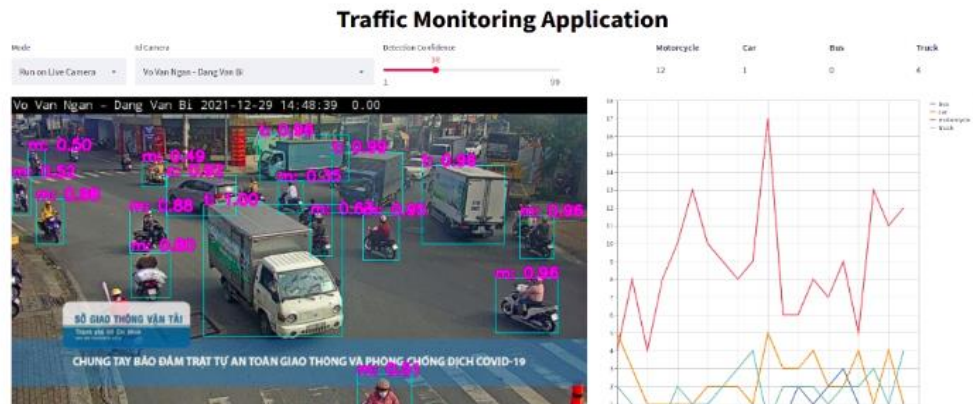
4.3. Detection Assessment

[license_plate_assessment](#)

5. Demo Application using CV

Detect Vehicles

- From Image
- From Video
- From Live Camera



Visualize Information With Charts

Using Realtime Information To Draw Charts

THANK YOU!



Email: contact@tmainnovation.vn



Website: www.tmainnovation.vn

